

Solution 1

Solution 1 (a)

Online Algorithm Implementation: Mean

```

1  # load libraries
2  import numpy as np
3  import matplotlib.pyplot as plt
4  import seaborn as sns
5
6  # quick check data-1.txt has 5000 pts
7  # cat data-1.txt | wc -l -> 5000
8
9  # load data-1.txt
10 data = np.loadtxt('data-1.txt')
11
12 def online_algorithm_mean(data):
13     mean_i = []
14     dt = []
15     current_mean = 0
16     for t,x_t in enumerate(data):
17         dt.append(t+1)
18         current_mean = current_mean + ((x_t-current_mean)/(t+1))
19         mean_i.append(current_mean)
20     print(f"final mean: {mean_i[-1]}")
21     return dt, mean_i
22
23 def plot_mean_dt(dt,mean_i,true_mean):
24     fig,ax = plt.subplots(nrows=1,ncols=1,figsize=(8,10))
25     sns.lineplot(x=dt,y=mean_i,markers='o',ax=ax,color='blue',label='OA')
26     sns.lineplot(x=dt,y=[true_mean for i in
27         dt],markers='--',ax=ax,color='red',label='True Mean')
28     ax.set_xlabel('dt')
29     ax.set_ylabel('mean')
30     ax.set_ylim(min(mean_i)-1e5,true_mean+1e5)
31     print(min(mean_i))
32     title = f"Online Algorithm(OA) for Mean\n True Mean: {true_mean:.3f} \n OA Mean:
33         {mean_i[-1]:.3f}"
34     ax.set_title(title)
35     fig.savefig('hw3_q1.png')
36
37 # calc true mean and print it
38 true_mean = sum(data)/len(data)
39 print(true_mean)
40
41 dt, mean_i = online_algorithm_mean(data)
42 plot_mean_dt(dt,mean_i,true_mean)

```

Comments

The true mean is 2,991,834.673 and is identical to the final mean calculated via `online_algorithm_mean(data)` function.

Solution 1

Solution 1 (b)

Plot

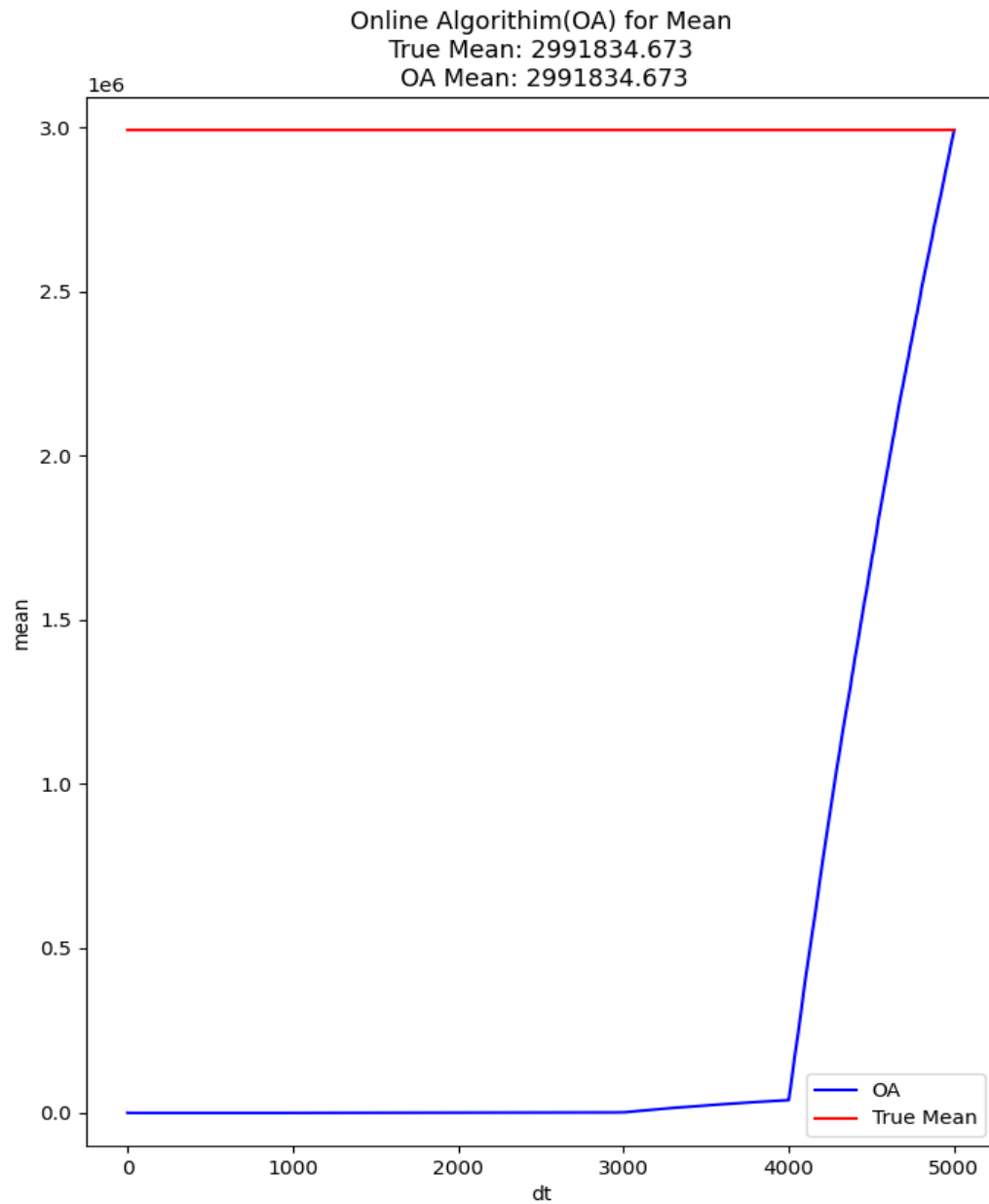


Figure 1: Figure shows all 5000 intermediate values of mean

Solution 2

Solution 2 (a)**Pseudocode**

Algorithm 1 Online Variance Calculation

Initialize:

$t \leftarrow 0$	▷ Count of data points
$\mu \leftarrow 0.0$	▷ Running mean of X
$M_2 \leftarrow 0.0$	▷ Running mean of X^2
$v \leftarrow 0.0$	▷ Running variance

For each new data point x_t :

$t \leftarrow t + 1$	
$\mu \leftarrow \mu + (x_t - \mu)/t$	▷ Update mean of X
$M_2 \leftarrow M_2 + (x_t^2 - M_2)/t$	▷ Update mean of X^2
$v \leftarrow M_2 - \mu^2$	▷ Calculate current variance

Solution 2

Solution 2 (b)

Online Algorithm Implementation: Variance

```
1  # load libraries
2  import numpy as np
3
4  # load data-1.txt
5  data = np.loadtxt('data-1.txt')
6
7  def online_algorithm_variance(data):
8      mean_x = 0.0
9      mean_x_squared = 0.0
10     variances = []
11
12     for t,x_t in enumerate(data):
13         # Update the mean of x
14         mean_x = mean_x + ((x_t-mean_x)/(t+1))
15         # Update the mean of x-squared
16         mean_x_squared = mean_x_squared + ((x_t**2-mean_x_squared)/(t+1))
17
18         variances.append(mean_x_squared - mean_x**2)
19
20     print(f"final variance: {variances[-1]:.3f}")
21     return variances
22
23 # calc true variance and print
24 true_variance = np.var(data)
25 print(f"true variance: {true_variance:.3f}")
26
27 # calc online_algorithm_variance
28 variances = online_algorithm_variance(data)
```

Comments

The true variance using numpy: 36,618,966,207,041.938 while the variance calculated via `online_algorithm_variance(data)` was: 36,618,966,207,041.961. This discrepancy between the true and *online variance calculation* arises because the online formula is sensitive to floating point precision errors, especially when subtracting two large, similar numbers.

Solution 3

Solution 3 (a)

True Median

```
1 # load libraries
2 import numpy as np
3
4 # load data-1.txt
5 data = np.loadtxt('data-1.txt')
6
7 # calc true median
8 true_median = np.median(data)
9 print(f"true median: {true_median:.3f}")
```

The true median using numpy: 1,526

Solution 3

Solution 3 (b)/(c)

Random Sample with Replacement Algorithm Implementation: Median

```

1  # load libraries
2  import numpy as np
3  import random
4
5  # load data-1.txt
6  data = np.loadtxt('data-1.txt')
7
8  def estimate_online_median(data, k, repetitions):
9
10     print(f"--- Starting estimations for k={k} ---")
11     for rep in range(repetitions):
12         # initialize the sample array 's' with the first data point.
13         # from s1=s2=...=sk=x1
14         s = np.full(k, data[0])
15
16         # get x_t from t=2,3,...,t
17         for i in range(1, n):
18             t = i + 1
19             x_t = data[i]
20
21             # for j=1,2,...,k then sj = xt with probability 1/t.
22             for j in range(k):
23                 if random.random() < (1.0 / t):
24                     s[j] = x_t
25
26         # calculate the median of the final sample for each repetition
27         estimated_median = np.median(s)
28         print(f"repetition {rep + 1:2d}: estimated median = {estimated_median}")
29     print("-" * (35 + len(str(k))))
30
31 # part (a): calc and print the true median
32 true_median = np.median(data)
33 print("--- true median calculation ---")
34 print(f"true median: {true_median}\n")
35
36 # part (b): simulation for k=100
37 estimate_online_median(data, k=100, repetitions=10)
38
39 # part (c): simulation for k=500
40 estimate_online_median(data, k=500, repetitions=10)
41

```

Solution 3

Solution 3 (b)/(c)

Random Sample with Replacement Algorithm Results: Median

(a) Estimated Medians for $k = 100$		(b) Estimated Medians for $k = 500$	
Repetition	Estimated Median	Repetition	Estimated Median
1	1750.0	1	1422.0
2	1094.0	2	1498.5
3	1236.0	3	1404.5
4	1166.0	4	1328.5
5	1552.5	5	1592.5
6	1338.5	6	1452.5
7	1583.5	7	1520.5
8	1635.0	8	1444.5
9	1805.5	9	1541.5
10	1376.0	10	1567.0

Table 1: Comparison of estimated medians for different sample sizes. *Note: as k grows sufficiently large the calculation for the median of a sample using the random sample with replacement should converge to the true median.*

Solution 4

An ϵ -heavy hitter is an element in a data stream of length m whose frequency is greater than or equal to ϵm . We need to find the maximum number of such elements when $\epsilon = 0.05$.

Let k be the number of distinct ϵ -heavy hitters. Let the frequency of the i -th heavy hitter be denoted by f_i .

By definition, for each of the k heavy hitters:

$$f_i \geq \epsilon m$$

The sum of frequencies of these distinct items cannot exceed the total stream length m :

$$\sum_{i=1}^k f_i \leq m$$

Substitute the minimum frequency for each heavy hitter into the sum:

$$\begin{aligned} \sum_{i=1}^k (\epsilon m) &\leq \sum_{i=1}^k f_i \leq m \\ k \cdot \epsilon \cdot m &\leq m \end{aligned}$$

Since $m > 0$, we can divide both sides by m :

$$k \cdot \epsilon \leq 1$$

Solving for k :

$$\begin{aligned} k &\leq \frac{1}{\epsilon} \\ k &\leq \frac{1}{0.05} \\ k &\leq 20 \end{aligned}$$

$\therefore k \leq 20$ demonstrates that for $\epsilon = 0.05$, there can be at most 20 ϵ -heavy hitters in the stream.

Solution 5

Solution 5 (a)**Optimal Clusters**

The k-means objective is to find a set of k centers $M = \{\mu_1, \dots, \mu_k\}$ that minimizes the sum of squared Euclidean distances from each data point to its nearest center. By inspection, the optimal partitioning separates the negative numbers, the positive numbers, and the origin.

Hence, the optimal clusters are $C_1^* = \{-10, -8\}$, $C_2^* = \{0\}$, and $C_3^* = \{8, 10\}$.

Optimal Centers

The optimal centers μ_j^* are calculated as the mean of the points in each optimal cluster C_j^* :

$$\begin{aligned}\mu_1^* &= \frac{-10 + (-8)}{2} = -9 \\ \mu_2^* &= \frac{0}{1} = 0 \\ \mu_3^* &= \frac{8 + 10}{2} = 9\end{aligned}$$

Hence, the optimal centers are:

$$\mu_1^* = -9 \quad \mu_2^* = 0 \quad \mu_3^* = 9$$

Optimal Cost

The globally optimal cost, J^* , is the sum of squared distances for this configuration:

$$\begin{aligned}J^* &= \sum_{x \in C_1^*} (x - \mu_1^*)^2 + \sum_{x \in C_2^*} (x - \mu_2^*)^2 + \sum_{x \in C_3^*} (x - \mu_3^*)^2 \\ &= ((-10 - (-9))^2 + (-8 - (-9))^2) + ((0 - 0)^2) + ((8 - 9)^2 + (10 - 9)^2) \\ &= ((-1)^2 + 1^2) + 0 + ((-1)^2 + 1^2) \\ &= (1 + 1) + 0 + (1 + 1) \\ &= 4\end{aligned}$$

$\therefore$ The optimal centers are $\{-9, 0, 9\}$ with a minimum cost of $J^* = 4$.

Solution 5**Solution 5 (b)**

Let

$$\mu_1 = -10 \quad \mu_2 = -8 \quad \mu_3 = 0 \quad \text{and} \quad k = 3$$

Assignment

Assign each point in X to the closest mean.

$$x_1 = -10 \rightarrow \mu_1 \text{ is closest} \rightarrow C_1 = \{-10\}$$

$$x_2 = -8 \rightarrow \mu_2 \text{ is closest} \rightarrow C_2 = \{-8\}$$

$$x_3 = 0 \rightarrow \mu_3 \text{ is closest} \rightarrow C_3 = \{0\}$$

$$x_4 = 8 \rightarrow \mu_3 \text{ is closest} \rightarrow C_3 = \{0, 8\}$$

$$x_5 = 0 \rightarrow \mu_3 \text{ is closest} \rightarrow C_3 = \{0, 8, 10\}$$

Hence, the resulting clusters are: $C_1 = \{-10\}$ $C_2 = \{-8\}$ $C_3 = \{0, 8, 10\}$

Update Centers

Recalculate centers as means of the new clusters:

$$\mu_1^{(1)} = \frac{-10}{1} = -10 \quad \mu_2^{(1)} = \frac{-8}{1} = -8 \quad \mu_3^{(1)} = \frac{0 + 8 + 10}{3} = 6$$

Hence, the new centers are: $M^{(1)} = \{-10, -8, 6\}$

Second Assignment

Assign each point in X to the closest mean.

$$x_1 = -10 \rightarrow \mu_1 \text{ is closest} \rightarrow C_1 = \{-10\}$$

$$x_2 = -8 \rightarrow \mu_2 \text{ is closest} \rightarrow C_2 = \{-8\}$$

$$x_3 = 0 \rightarrow \mu_3 \text{ is closest} \rightarrow C_3 = \{0\}$$

$$x_4 = 8 \rightarrow \mu_3 \text{ is closest} \rightarrow C_3 = \{0, 8\}$$

$$x_5 = 0 \rightarrow \mu_3 \text{ is closest} \rightarrow C_3 = \{0, 8, 10\}$$

Final Centers

Since the cluster assignments did not change, the centers will not change either.

Hence, the final centers are: $M^{(F)} = \{-10, -8, 6\}$

Final Cost

$$\begin{aligned} J^F &= ((-10 - (-10))^2) + ((-8 - (-8))^2) + ((0 - 6)^2 + (8 - 6)^2 + (10 - 6)^2) \\ &= 0 + 0 + ((-6)^2 + 2^2 + 4^2) \\ &= 36 + 4 + 16 \\ &= 56 \end{aligned}$$

$\therefore$ The algorithm terminates at the centers $\{-10, -8, 6\}$ with a cost of $J^F = 56$.

Solution 6

Solution 6 (a)

Dendrogram

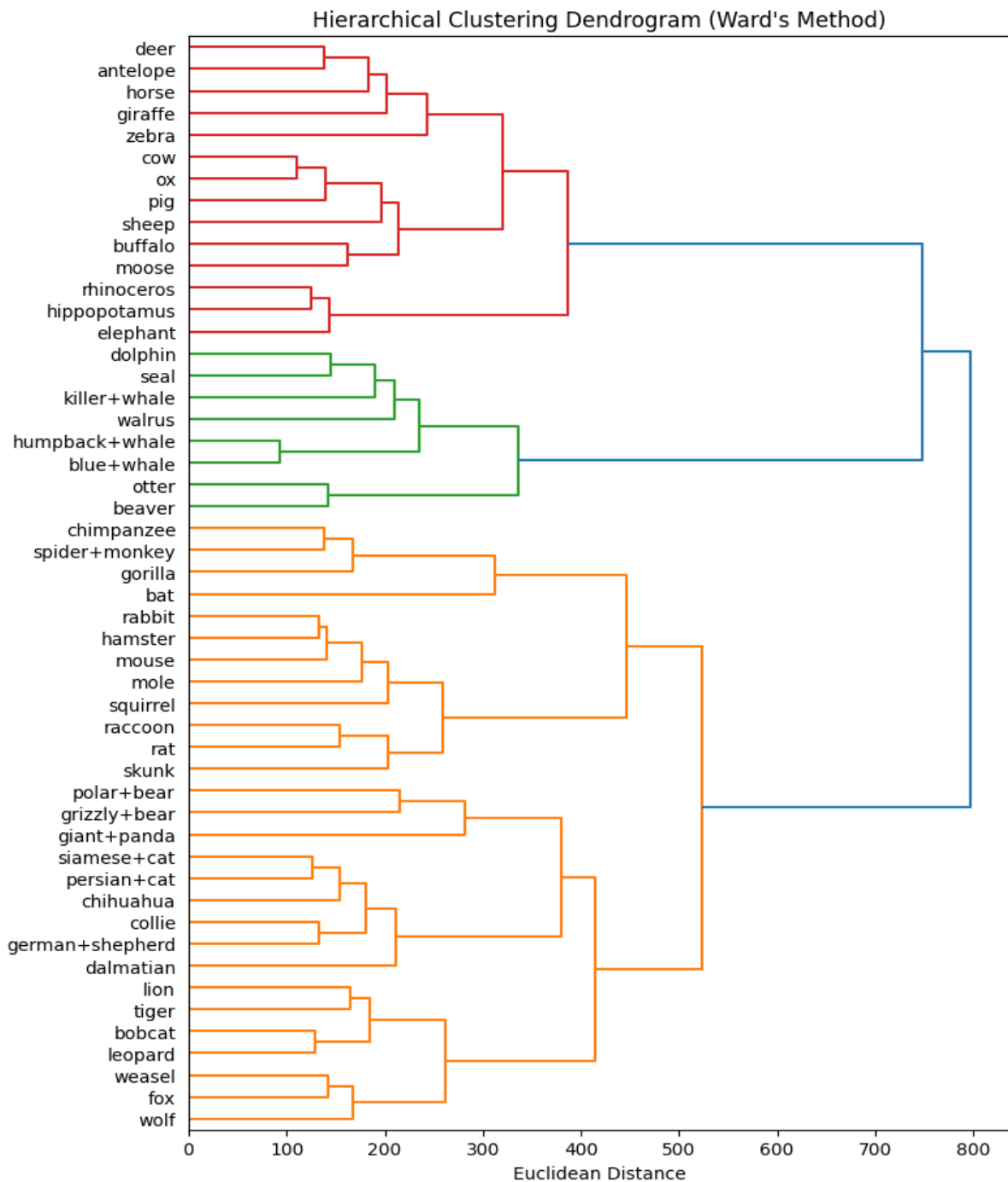


Figure 2: Figure shows the animal dendrogram.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.cluster import hierarchy

```

```
4 from sklearn.cluster import KMeans
5 from collections import defaultdict
6
7 # set plot figure size for better readability of the dendrogram
8 plt.rcParams['figure.figsize'] = (8, 10)
9
10 # read predicate-matrix-continuous.txt as numpy array
11 X = np.loadtxt('predicate-matrix-continuous.txt')
12
13 # read classes.txt to create animal name list
14 animal_names = []
15 with open('classes.txt', 'r') as f:
16     for line in f:
17         parts = line.strip().split()
18         # The name is the second part, possibly with '+'
19         animal_names.append(parts[1])
20
21 # perform hierarchical clustering using Ward's linkage method
22 Z = hierarchy.linkage(X, method='ward')
23
24 # plot the dendrogram
25 plt.figure()
26 hierarchy.dendrogram(
27     Z,
28     labels=animal_names,
29     orientation='right',
30     leaf_font_size=10
31 )
32 plt.title("Hierarchical Clustering Dendrogram (Ward's Method)")
33 plt.xlabel("Euclidean Distance")
34 plt.tight_layout()
35 plt.show()
```

Solution 6

Solution 6 (b)

Python Code

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  from scipy.cluster import hierarchy
4  from sklearn.cluster import KMeans
5  from collections import defaultdict
6
7  # read predicate-matrix-continuous.txt as numpy array
8  X = np.loadtxt('predicate-matrix-continuous.txt')
9
10 # read classes.txt to create animal name list
11 animal_names = []
12 with open('classes.txt', 'r') as f:
13     for line in f:
14         parts = line.strip().split()
15         # The name is the second part, possibly with '+'
16         animal_names.append(parts[1])
17
18 # perform hierarchical clustering using Ward's linkage method
19 Z = hierarchy.linkage(X, method='ward')
20
21 # get 10 clusters from the hierarchical model
22 ward_clusters = hierarchy.fcluster(Z, 10, criterion='maxclust')
23
24 # group animals by their assigned cluster
25 ward_cluster_dict = defaultdict(list)
26 for i, cluster_id in enumerate(ward_clusters):
27     ward_cluster_dict[cluster_id].append(animal_names[i])
28
29 # calculate the k-means cost (sum of squared distances) for this clustering
30 ward_cost = 0.0
31 centroids = {}
32 for i in range(1, 11):
33     # get all data points for the current cluster
34     cluster_points = X[ward_clusters == i]
35     # calculate the centroid (mean) of the cluster
36     centroid = np.mean(cluster_points, axis=0)
37     centroids[i] = centroid
38     # calculate sum of squared distances from points to the centroid
39     ward_cost += np.sum((cluster_points - centroid)**2)
40
41 print("wards clustering (k=10)")
42 for cid, animals in sorted(ward_cluster_dict.items()):
43     print(f"cluster {cid}: {' '.join(animals)}")
44 print(f"\nkmeans cost for wards clustering: {ward_cost:.4f}")
45
46 # run kmeans 10 times with different random initializations
47 kmeans = KMeans(n_clusters=10, n_init=10, random_state=42)
48 kmeans.fit(X)
49
50 # inertia_ attribute stores the ssd (kmeans cost) for the best of the 10 runs
51 kmeans_cost = kmeans.inertia_
52
53 print("\n--- kmeans algorithm (10 runs) ---")
54 print(f"best kmeans cost found: {kmeans_cost:.4f}")

```

Solution 6**Solution 6 (b)****Results**

Cluster	Method	Cost
1: tiger, leopard, fox, wolf, weasel, bobcat, lion 2: dalmatian, persian cat, german shepherd, siamese cat, chihuahua, collie 3: grizzly bear, giant panda, polar bear 4: skunk, mole, hamster, squirrel, rabbit, rat, mouse, raccoon 5: spider monkey, gorilla, chimpanzee, bat 6: beaver, otter 7: killer whale, blue whale, humpback whale, seal, walrus, dolphin 8: hippopotamus, elephant, rhinoceros 9: moose, ox, sheep, buffalo, pig, cow 10: antelope, horse, giraffe, zebra, deer	Ward's Method	679 136.2673
Best K-means Cost found (10 runs)	K-means	707 405.1627

Table 2: Comparison of Clustering Results for $k = 10$ **Solution 7****Solution 7 (a)****Optimal Centers**

To find the optimal centers for $k = 3$, we must inspect the structure of the dataset. By inspection, we see that there exists three distinct groups:

1. The point $\begin{bmatrix} 1 \\ 8 \end{bmatrix}$.
 - (a) One center, μ_1^* , covers the outlier $\begin{bmatrix} 1 \\ 8 \end{bmatrix}$. The optimal placement is on the point itself.
2. A square-like cluster.
 - (a) A second center, μ_2^* , covers the cluster $\left\{ \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \begin{bmatrix} 2 \\ 2 \end{bmatrix} \right\}$. The center of the minimal enclosing circle is their geometric mean, $\begin{bmatrix} 1.5 \\ 1.5 \end{bmatrix}$.
3. a horizontal line segment.
 - (a) A third center, μ_3^* , covers the segment $\left\{ \begin{bmatrix} 5 \\ 1 \end{bmatrix}, \begin{bmatrix} 6 \\ 1 \end{bmatrix} \right\}$. The optimal placement is the midpoint, $\begin{bmatrix} 5.5 \\ 1 \end{bmatrix}$.

Hence, the optimal centers are:

$$M^* = \{\mu_1^*, \mu_2^*, \mu_3^*\} = \left\{ \begin{bmatrix} 1 \\ 8 \end{bmatrix}, \begin{bmatrix} 1.5 \\ 1.5 \end{bmatrix}, \begin{bmatrix} 5.5 \\ 1 \end{bmatrix} \right\}$$

Optimal Cost

Calculate the radius from the center to the farthest point in each cluster since we are in a $2D$ space.

$$\begin{aligned} \text{radius}_1 &= \left\| \begin{bmatrix} 1 \\ 8 \end{bmatrix} - \begin{bmatrix} 1 \\ 8 \end{bmatrix} \right\|_2 = 0 \\ \text{radius}_2 &= \left\| \begin{bmatrix} 1 \\ 1 \end{bmatrix} - \begin{bmatrix} 1.5 \\ 1.5 \end{bmatrix} \right\|_2 = \sqrt{(-0.5)^2 + (-0.5)^2} = \sqrt{0.5} \\ \text{radius}_3 &= \left\| \begin{bmatrix} 5 \\ 1 \end{bmatrix} - \begin{bmatrix} 5.5 \\ 1 \end{bmatrix} \right\|_2 = \sqrt{(-0.5)^2} = 0.5 \end{aligned}$$

The cost is the maximum of these radii above, so $J^*(M^*) = \max(0, \sqrt{0.5}, 0.5) = \sqrt{0.5}$.

$\therefore$ The optimal centers are $\left\{ \begin{bmatrix} 1 \\ 8 \end{bmatrix}, \begin{bmatrix} 1.5 \\ 1.5 \end{bmatrix}, \begin{bmatrix} 5.5 \\ 1 \end{bmatrix} \right\}$ with a minimum cost of $J^* = \sqrt{0.5}$.

Solution 7**Solution 7 (b)****Select μ_2**

Find the point in X that is farthest from the initial center $\mu_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$.

$$d^2 \left(\begin{bmatrix} 1 \\ 8 \end{bmatrix}, \mu_1 \right) = (1-1)^2 + (8-1)^2 = 49 \quad (\text{Max})$$

$$d^2 \left(\begin{bmatrix} 6 \\ 1 \end{bmatrix}, \mu_1 \right) = (6-1)^2 + (1-1)^2 = 25$$

$$d^2 \left(\begin{bmatrix} 5 \\ 1 \end{bmatrix}, \mu_1 \right) = (5-1)^2 + (1-1)^2 = 16$$

$$d^2 \left(\begin{bmatrix} 2 \\ 2 \end{bmatrix}, \mu_1 \right) = (2-1)^2 + (2-1)^2 = 2$$

The farthest point is $\begin{bmatrix} 1 \\ 8 \end{bmatrix}$, so $\mu_2 = \begin{bmatrix} 1 \\ 8 \end{bmatrix}$. The current centers are $M_2 = \left\{ \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 8 \end{bmatrix} \right\}$.

Select μ_3

Find the point $x \in X \setminus M_2$ that maximizes $\min_{\mu \in M_2} \|x - \mu\|_2$. To check this condition we must look at the minimum squared distance for each remaining point to the set M_2 .

$$x = \begin{bmatrix} 6 \\ 1 \end{bmatrix} : \min(d^2(x, \mu_1) = 25, d^2(x, \mu_2) = 74) = 25 \quad (\text{Max})$$

$$x = \begin{bmatrix} 5 \\ 1 \end{bmatrix} : \min(d^2(x, \mu_1) = 16, d^2(x, \mu_2) = 65) = 16$$

$$x = \begin{bmatrix} 2 \\ 2 \end{bmatrix} : \min(d^2(x, \mu_1) = 2, d^2(x, \mu_2) = 37) = 2$$

$$x = \begin{bmatrix} 2 \\ 1 \end{bmatrix} : \min(d^2(x, \mu_1) = 1, d^2(x, \mu_2) = 50) = 1$$

Hence, the point maximizing this minimum distance is: $\begin{bmatrix} 6 \\ 1 \end{bmatrix}$, so $\mu_3 = \begin{bmatrix} 6 \\ 1 \end{bmatrix}$.

Final Centers

Hence, the final set of centers is:

$$M^{(F)} = \left\{ \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 8 \end{bmatrix}, \begin{bmatrix} 6 \\ 1 \end{bmatrix} \right\}$$

Solution 7

Solution 7 (b)**Final Cost**

The cost $J^{(F)}$ is the maximum distance from any point in X to its nearest center in $M^{(F)}$. Now, find the maximum of the minimum squared distances for all non-center points.

$$d_{\min}^2 \left(\begin{bmatrix} 2 \\ 2 \end{bmatrix}, M^{(F)} \right) = \min \left(d^2 \left(\begin{bmatrix} 2 \\ 2 \end{bmatrix}, \mu_1 \right) = 2, d^2 \left(\begin{bmatrix} 2 \\ 2 \end{bmatrix}, \mu_2 \right) = 37, d^2 \left(\begin{bmatrix} 2 \\ 2 \end{bmatrix}, \mu_3 \right) = 17 \right) = 2 \quad (\text{Max})$$

$$d_{\min}^2 \left(\begin{bmatrix} 5 \\ 1 \end{bmatrix}, M^{(F)} \right) = \min(16, 65, 1) = 1$$

The maximum squared distance is 2. The cost is the square root of this value.

$$J^{(F)}(M^{(F)}) = \sqrt{2}$$

$\therefore$ the centers are: $\left\{ \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 8 \end{bmatrix}, \begin{bmatrix} 6 \\ 1 \end{bmatrix} \right\}$ with a cost of $J^{(F)} = \sqrt{2}$.
